

20XX 年度 修士論文

修論テンプレ

(Template for Master thesis)

東京大学理学系研究科
物理学専攻 XX 研究室
学籍番号 xxxxxxxxxx

おなまえ

2026 年 9 月 11 日

概要

これは概要

タイトルページについては何書くとか配置とかが所属によって流派が分かれると思うので、ここは特にテンプレ化はせず単にテキストを place で配置するだけにしている。変更するには main.typ の最初の[]のブロックをいじれば良い。

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

main.typ に直接書き込んだって構わない(←け○た食堂)

目次

第 1 章	はじめに	1
1.1	how to use	1
1.2	ファイル構造	2
第 2 章	基本の記法	3
2.1	==だと level2	3
2.1.1	===だと level3	3
2.1.2	ほらね	3
2.1.3	こいつには適用されない。	3
2.2	figure	3
2.2.1	image	4
2.2.2	table	7
2.2.3	raw	7
2.2.4	figure のバグ	8
2.3	数式	8
2.3.1	tex と比較(数式)	9
2.4	リンク	10
2.5	reference	10
2.6	par	11
2.7	page	11
第 3 章	tips	13
3.1	有用な関数	13
3.1.1	mymemo, TBC	13
3.1.2	todo, show_todo	13
3.1.3	pbox	13
3.1.4	smaller_list	14
3.1.5	math-ul	15
3.2	有用なパッケージ	15
3.2.1	unify, physica, fletcher	15
3.2.2	zero	15
3.2.3	cheq	16
Appendix		18
A	応用	19
A.1	目次いじり	19
A.1.1	チャプターごとにスペース+点線一部削除	19
A.1.2	図 1.1 → 1.1	20
A.2	footnote いじり	21
B	dictionary	23
謝辞		26
参考文献		27

図目次

図 1.1	preview のボタン位置	2
図 2.1	カービイじゃねえカービイだにどとまちがえるなくそが	4
頭 2.2	カービイじゃねえカービイだにどとまちがえるなくそが	6
図 2.3	図目次用短いキャプション	6
図 2.4	placement: bottom	8
図 2.5	place+figure	8
図 A.1	appfig_test	22

表目次

表 2.1	データ型	7
表 3.1		16
表 3.2		16
表 A.1	apptable_test	22
表 B.1		24

第 1 章

はじめに

typst は 3,4 年前くらいに公開された新しめの組版システム。正しい発音は /taɪpst/, つまり「タイプスト」である。主な特徴として

- オープンソースで盛んに開発が進められている
- 環境構築が非常に楽。
 - ▶ homebrew を使う場合は `brew install typst` で数分待つて終わり。tex とは大違いですな
- tex よりコンパイルが速い。
 - ▶ Rust 製なので普通に速いのと、基本が差分コンパイルなので tex みたく毎回 1 から 10 までコンパイルし直すようなことがない。でかいファイルを作る場合にはこれが重宝する。
- tex よりエラー箇所の指摘がわかりやすい。
- 結構自由にカスタム可能。人の作ったテンプレを持ってくるなども結構容易(これは tex もそうだったかな)

あたりがパツと思ひ浮かぶ。この辺はメリットを挙げたが、逆にデメリットとしては

- tex に比べ日本語の文献が少ない
 - ▶ 一応ドキュメントは結構充実しているので、それを読み込めば大抵のことはできるっちゃできる。
- tex にあるが typst には標準装備してない機能とかまあまあある
 - ▶ issue 見ればすでに議論されてて対処法が書いてあるなんてこともちらほら。
 - ▶ 逆に個人的な拘りで特殊なことをやろうとすると自力で頑張る必要があったりなかったりくらいでしょーか。年が浅いことに由来するデメリットが考えつくが、まあ結構なスピード感でアップデートされているものなので、また数年経てば状況は変わっているのではないのでしょうか、と個人的には思う。

ここでは基本的に修論を書くための事項を挙げていくが、その他単なる研究ノートとしてだったりパワポに代わる発表スライドとしても使える。気が向いたらそっちのテンプレについてもまとめるかもしれない。

1.1 how to use

上でも触れた通り、homebrew を使ってれば一発。そのほかの場合は知らないので調べてほしいが、いずれにせよ tex より圧倒的に楽に済む。

インストールしてしまえばもうすぐ使える状態になるが、追加で tinymist という言語サーバを加えておくと非常に便利が良い。筆者のようにメインのエディタとして vscode を使ってる場合は同名の拡張機能があるので入れておこう。vim などの場合はちょっと様相が異なるが、これもやり方をまとめる人いたので各自調べてください。

基本的には適当にコード書いて `typst compile input.typ output.pdf` を叩けば爆速コンパイルの `output.pdf` が生成される。ただこのコマンドはそこまで使っていない。

一つ目の上位互換として `typst watch input.typ output.pdf` があり、こいつを叩くとコードをいじってセーブするたびに差分コンパイルを実行して pdf を更新してくれる。いちいちコマンドを叩きにターミナルへ移動する必要がなくなるのだ。

そして筆者が主に使っていたのが tinymist の機能である preview。vscode では [図 1.1](#) に示したボタンをクリックしたら vscode の別タブとして開かれる。chrome など普段使っているブラウザのタブに展開することもできる。vim なら `:TypstPreview` でブラウザに開いてくれる。typst watch がコードを変更後保存したタイミングでコンパイルするのに対し、こいつは書いた側から変更が反映

される文字通りのリアルタイムプレビューができる点が強み。レイアウトを考えながら書く時なんかには大変重宝する¹。



図 1.1: preview のボタン位置

あと、web 上で文書作成できるサービスもある(<https://typst.app/>)。tex の overleaf 的な立ち位置なのだろうか、筆者は使ってないのでよくわからない。

1.2 ファイル構造

テンプレ作成に使ったファイル群を紹介しておく。

- main.typ
その名の通りメインのファイル。タイトルページの編集、使用するチャプターファイルの順番指定を行なっている。こいつをコンパイルすることで最終版を出力できる。
- theme/state.typ
元々 state というタイプの変数をまとめる役割だったが、state はあまり使わない方がロバストなテンプレを作れるということで、in-outline を除き state のない state.typ が出来上がった。適切に rename してないのはなんとなく。タイトル、概要-目次、本文、付録、謝辞、参考文献それぞれについての設定をまとめている。outline の設定部分とかもう少しスマートにできそうなもんだがまあ一旦このくらいでいいかなと思っている。
- theme/template.typ
各ファイルで import している、お役立ち関数・変数まとめファイル。state.typ で使用するテンプレに加え、筆者が修論執筆時に使用した関数を適当に残している。文書全体で使いたい便利関数をここで定義して、各 typ ファイル冒頭にて `#import "../template.typ: *"` してやれば各ファイル内で使えるようになる。本テンプレをさらにカスタムしたい場合は基本的に state.typ とこいつのいずれかをいじればおけ。
- theme/header_components.typ
ヘッダーに関する関数まとめ。チャプターページか否か/ページの偶奇に応じてヘッダー表示を変えたかったのでこういうのが必要になった。別に全ページ同じでよくね、ってなるのであれば、単に state.typ 内(あるいは main-template など)で `#set page` してやればよいのでこいつは不要。
- chapter/chapter*.typ
チャプターごとにファイルを分けるためにおいてる。分ける必要がないと思うのであれば最初から main.typ に書き込んでしまってもよい。使い方はコード見ればなんとなくわかるだろう。
- references.bib
参考文献として使う bibtex ファイル。使い方は tex でやるのと全く同様なので割愛。

¹ちなみに vscode の tinymist はコマンド叩かずともセーブしたら勝手に同名の pdf をコンパイルしてしまう。json をいじればこの辺変更できるが、筆者はまあどうでもええなと思って放置している。

第 2 章

基本の記法

本章では基本的な文書の書き方を簡単にまとめる。本テンプレを使うだけなら、この章を読めば最低限のものは作れるようになるはず。もちろん見た目をこだわりたいとかあればその限りではない。

基本は markdown 形式で文章を書ける。= の数で heading のレベルを指定してセクション分けをしよう。

2.1 ==だと level2

2.1.1 ===だと level3

2.1.1.1 ====で level4 と続けてもいいがまあ普通は 3 までよな

各 heading の文字サイズは

```
#show heading.where(level: 3): set text(50pt)
```

とかすればレベル別に指定可能。

2.1.2 ほらね

こういう show ルール、set ルールを適用すると普通は以下全てのコンテンツに適用されてしまう。今の場合見出し 2.1.2 だけ 50pt で表示したければ、

```
#[
  #show heading.where(level: 3): set text(50pt)
  === ほらね
]
```

こうやってコンテンツブロックで囲ってやれば、ルールがそのブロック内でのみ適用されるので便利。

2.1.3 こいつには適用されない。

ちなみに今現在の heading の文字サイズ設定は template.typ の main-template に記述してある。大きさに不満があるならそこをいじってください。

2.2 figure

図は #image、表は #table、コードは #raw を使って表示可能。だが基本的には figure の中身に記述するようにしよう。

2.2.1 image

図を挿入するには

```
#image("../fig/test.jpg", width: 50%)
```

と書く。



←かわいい

width はそのまま、画像の幅。50% という指定はページ幅に対する半分サイズってこと。height で指定することも可能。

ただのノートくらいならこれでいいだろうが、論文にする場合は #figure を使うべし。

```
#figure(
  image("../fig/test.jpg"),
  caption: [カービィじゃねえカービィだにどとま
    ちがえるなくそが]
)<kirby_fig>
```



図 2.1: カービィじゃねえカービィだにどとま
ちがえるなくそが

caption がつけられるようになるほか、reference が可能となる。<kirby_fig>とタグをつけたので、@kirby_fig と書けば図 2.1 のように自動で画像位置にリンクをつけられる。あと普通に、figure を使わず単に image してしまうと冒頭の図目次に反映されない¹。

ちなみに、使ったついでに言うと今の 2 列表示は #grid でやってます。下の例で columns を 2, rows を指定しなければよい。

```
#grid(
  columns: 3, // 単に行数/列数で記述もできるし
  rows: (20%, 10%), // 比率で記述することも可能
  stroke: red,
  inset: 5pt // コンテンツと枠線の距離を指定できる。5pt以上入れといた方が見やすい。
[
  左上ブロック
],
[
  #image("../fig/test.jpg"),
],
[
  右上ブロック\
  各コンテンツブロックは独立なので、例えば`#set align(right)`を叩いても\
```

¹もしかしたら筆者が知らないだけで、figure だけでなく image に対して outline できる方法あるのかもだが、今のところ真面目な文書で figure にしない理由が筆者には思い当たらないので特に理由なければこの運用をすることをお勧めします

```
#v(1em)
#set align(right)
    ここだけ右詰め of 文書になり、後ろのブロックは普通に左詰めになる。
],
[
    左下ブロック\
    問題なく左詰め
],
[
    中央下ブロック\
    `rows: (20%, 10%),`としたので、1行目、2行目の高さがページに対する20%, 10%になっている。
],
[
    右下ブロック
]
)
```

左上ブロック	ここで <code>#image("../fig/test.jpg", width: 50%)</code> と書いた場合、このセルの幅に対する 50% で描画することになる。 	右上ブロック 各コンテンツブロックは独立なので、例えば <code>#set align(right)</code> を叩いても ここだけ右詰め of 文書になり、後ろのブロックは普通に左詰めになる。
左下ブロック 問題なく左詰め	中央下ブロック <code>rows: (20%, 10%),</code>としたので、1行目、2行目の高さがページに対する 20%, 10% になっている。	右下ブロック

わかりやすさのために stroke を red にしたよ。

なお、(heading や table などともそうだが)例えば今「図 1.1」とあるのを「figure 1.1」などに変えたい場合は、set ルールで figure の supplement 引数をいじればよい。

```
#set figure(supplement: [頭])
#figure(
    image("../fig/test.jpg", width: 50%),
    caption: [カービイじゃねえカービィだにどとまちがえるなくそが],
)
```



頭 2.2: カービィじゃねえカービィだにどとまちがえるなくそが

あと、まだ typst が標準装備してない便利機能としてキャプションを本文と図見出しとで変える機能がある。これは今のところ `flex_caption` というのを実装して対応している。

```
// template.typにあるぞい
#let in-outline = state("in-outline", false)
#let flex_caption(short, long) = context {
  if in-outline.get() [#short] else [#long]
}
```

`state` 関数はそのまま状態を保持するもの。template.typ においては一番外側のスコープで定義したので、実質(template.typ を import した)文書内のどの場所にでもその状態を持ってきて使用できる。flex_caption がやっているのは in-outline が true かどうかで吐き出すコンテンツを受け取る引数のどちらにするかを決めるだけである。そして、state.typ にて

```
#show outline: it => {
  in-outline.update(true)
  it
  in-outline.update(false)
}
```

と書いておくことで、outline を表示する際先に in-outline が true になってから出力、その後は false に戻してから終了するように設定している。flex_caption とこの show ルールを組み合わせることで、図目次を出力する際は #short を、outline を表示し終えた本文中では #long が出力されるという仕組み。使い方としては caption の中身に記述するだけ。

```
#figure(
  image("../fig/test.jpg", width: 50%),
  caption: flex_caption(
    [図目次用短いキャプション],
    [本文用長いキャプション],
  )
)
```



図 2.3: 本文用長いキャプション

尚このテンプレは元々 state を大量に使っていたが、より明確な設計にするためにできるだけ state を使わないようにしようという思想のもと出来たものなのでここ以外では登場しない。flex_caption は state なしで実装できるのかしら。知らんし 1 個くらい許せよって感じなのでそのままにしてある。

ちなみに若干 off-topic だが、typst においては `hoge([a])` と `hoge()[a]` は同義である。つまり今の例で言えば

```
#figure(
  image("../fig/test.jpg", width: 50%),
  caption: flex_caption()[short][long]
)
```

のように書いても問題なく通る。好きな方を使えばよろし。

2.2.2 table

表は `#table` を使う。結構できることが多く全てを紹介するのは非常に面倒なので、基本的なことに留める(いうてこれ以外もだいたいそんな感じなのだが)。

`table` で主に用いる引数は `column`, `align`, `fill`, `stroke` あたり。それぞれ列数、セル内の文字位置、セルの色、罫線を指定する。[ドキュメント](#) に例がたくさんあるので、これをみながら遊ぶとよい。

```
#figure(
  table(
    columns: 3,
    stroke: none,
    align: {center},
    fill: (x,y) => {
      if x == 0 {aqua}
      else if calc.odd(y) {silver}
      else {none}
    },
    table.hline(),
    [型],[宣言],[ビット幅],
    table.hline(),
    [文字型],[char],[8],
    [整数型],[int],[32],
    [倍精度実数型],[double],[64],
    [倍々精度実数型],[long double],[96],
  ),
  caption: [データ型]
)
```

表 2.1: データ型

型	宣言	ビット幅
文字型	char	8
整数型	int	32
倍精度実数型	double	64
倍々精度実数型	long double	96

すげーな `typst`。table の場合は `caption` を上に置くってルールがあるけどそれも勝手にやってくれるのか。……いいえ、普通に `main-template` の中で `show` してるだけです

```
#show figure.where(kind: table): set figure.caption(position: top)
```

課題としては、今の所 `stroke` としての二重線がサポートされておらず、無理やりやるなら `column-gutter` でなんとかするしかないという点が挙げられる。これも早くアップデートしてほしい。

ところで、ふつー論文見たいなちゃんとした文章で書く `table` において全ての辺を書くことはまずないだろうし、いらないところを消すよりはいるところに `table.hline()`, `table.vline()` する方が見やすいと思うんで、いっそ `main-template` で合わせて `stroke: none` に `set` してしまってもいい気はする。

2.2.3 raw

`raw` コードは `#raw` でもいけるが大抵は ``raw code`` とバッククォートで囲んで表示する。バッククォート一つでインライン表記 `inline` になるし、三つでブロック表記

block

になる。今ブロックの時にグレーのブロックに収められた形で表示されてるのは `main-template` でそう記述しているからです

```
#show raw.where(block: true): it => {
  block(inset: 5pt, fill: silver.lighten(30%), radius: 3pt)[#it]
}
```

デフォルトでは特に `fill` されない。好みに合わせてこの辺の記述をいじるか消すかしてね。

`image`, `table` 同様これについても `outline` できるが、筆者の修論においては不要だったため特に記述してない。やりたければ `state.typ` の `outline` のところで `kind` を `raw` に指定すればおけ。

2.2.4 figure のバグ

一点、困ったことがある。figure には placement オプションがあり、none, top, bottom, auto から選べる。図 2.4 は top にしてみた結果であり、組版結果としては指示通り綺麗にできているように見えるが、このリンクを押してみると適切に画像位置に飛ばないことがわかる。どうやら figure がリンクしてくれる位置は組版結果としての出力位置ではなくコード内で figure を出力せよと指示された場所になるらしいのである。これは typst のバグ。普通に早く修正してほしい。

対処法としては figure に metadata を埋め込み、組版結果としての出力位置を保持しておき、ref する先をそれに塗り替えるよう show ルールを適用する、というのがあるらしいがなぜか自分はこれがうまく動かせなかったたので簡単に figure を place で囲む方式をとっている。

```
#place(auto, float: true)[
  #figure(
    image("../fig/test.jpg", width: 40%),
    caption: [place+figure]
  )
]
```

普通 place すると他のコンテンツから浮くのだが、float オプションを true にすることで浮かせることなく記述することができる。auto にするといふ感じに勝手にポジションを決めてくれるので便利。文章がある程度固まったら好みに合わせて適宜 top/bottom だの left/center/right だのすればよい。



図 2.5: place+figure

2.3 数式

数式を書くには \$\$ で囲めば良い。\$E = m c ^2\$ と書けばインラインで $E = mc^2$ となり、

```
$
x^n + y^n = z^n
$ <equation1>
```

と書けば block として

$$x^n + y^n = z^n \quad (2.1)$$

のように表示される。block にした equation については figure と同様に ref ができる。@equation1 で式 (2.1) という感じ。

便利かつ序盤は不便に感じる点として、tex とは異なりバックスラッシュとかなくとも基本文字入れるとまず変数として処理される。数式中でギリシャ文字とか使うには単に \$mu\$ とかしてすれば μ と出力される。慣れないうちは文字式 μ を出したいのに μ が出力されちゃうとか結構あると思う。普通に空白込みで \$m u\$ と書けばよいが、慣れるまでは若干苦勞するだろう。

数式番号については main-template にて以下のように記述した。

```
#set math.equation(numbering: it => {
  let count = counter(heading.where(level: 1)).at(here()).first()
  numbering("(1.1)", count, it)
```



図 2.4: placement: bottom

```

})
#show math.equation.where(block: true): set block(breakable: true)
#show math.equation.where(block: false): set math.frac(style: "horizontal")
#show math.equation: set text(font: ("New Computer Modern Math", "Hiragino Mincho ProN"))

```

上から順に

- ・ 該当チャプター内での通し番号 `it`, チャプター番号 `count` を用いて(`count`, `it`)という番号を振る、という規則を記述
 - ・ `it` がチャプター内通し番号になるのは、`heading_components.typ` にて `level1 heading` が来るたびに `counter(math.equation).update(0)` を叩くように指定したため。
- ・ ブロック表記の数式について、複数ページに分割して出力されることを許可
- ・ インライン表記の数式について、分数を a/b の形で出力する
- ・ 数式モード内でのフォント指定。日本語はヒラギノ明朝で意味している。

2.3.1 tex と比較(数式)

ちょっと脱線。typstの数式(に限らないが)はtexに比べてタイプ量が若干少なく済む。以前ついた一で「texでゼミ資料作るのが大変(>_<)」的なことを言ってる人がいたので、その人が書いてたコードを例に取って比較してみる。

$$\begin{aligned}
 i\hbar \frac{d\tilde{\psi}(\boldsymbol{x}, t)}{dt} &= i\hbar \frac{d}{dt} (\psi(\boldsymbol{x}, t) e^{-i\boldsymbol{K}(t) \cdot \boldsymbol{x}}) \\
 &= i\hbar \left[\left(\frac{d\psi(\boldsymbol{x}, t)}{dt} \right) e^{-i\boldsymbol{K}(t) \cdot \boldsymbol{x}} + \psi(\boldsymbol{x}, t) \frac{d}{dt} (e^{-i\boldsymbol{K}(t) \cdot \boldsymbol{x}}) \right] \\
 &= \left(i\hbar \frac{d\psi(\boldsymbol{x}, t)}{dt} e^{-i\boldsymbol{K}(t) \cdot \boldsymbol{x}} \right) + \psi(\boldsymbol{x}, t) \left(i\hbar e^{-i\boldsymbol{K}(t) \cdot \boldsymbol{x}} \left(-i \frac{d\boldsymbol{K}(t)}{dt} \cdot \boldsymbol{x} \right) \right) \\
 &= \left(i\hbar \frac{d\psi(\boldsymbol{x}, t)}{dt} \right) e^{-i\boldsymbol{K}(t) \cdot \boldsymbol{x}} + \psi(\boldsymbol{x}, t) e^{-i\boldsymbol{K}(t) \cdot \boldsymbol{x}} \left(\hbar \frac{d\boldsymbol{K}(t)}{dt} \cdot \boldsymbol{x} \right)
 \end{aligned} \tag{2.2}$$

こいつを出力する raw コードを以下に示す。括弧内は文字数(vscode によるカウント)。

tex(875)

```

\begin{align*}
i\hbar\frac{d\tilde{\psi}(\boldsymbol{x},t)}{dt}
&= i\hbar\frac{d}{dt} \left( \psi(\boldsymbol{x},t)e^{-i\boldsymbol{K}}(t)\cdot\boldsymbol{x} \right) \\
&= i\hbar \left[ \left( \frac{d\psi(\boldsymbol{x},t)}{dt} \right) e^{-i\boldsymbol{K}}(t)\cdot\boldsymbol{x} + \psi(\boldsymbol{x},t) \frac{d}{dt} \left( e^{-i\boldsymbol{K}}(t)\cdot\boldsymbol{x} \right) \right] \\
&= \left( i\hbar\frac{d\psi(\boldsymbol{x},t)}{dt} \right) e^{-i\boldsymbol{K}}(t)\cdot\boldsymbol{x} + \psi(\boldsymbol{x},t) \left( i\hbar e^{-i\boldsymbol{K}}(t)\cdot\boldsymbol{x} \left( -i\frac{d\boldsymbol{K}}{dt} \cdot \boldsymbol{x} \right) \right) \\
&= \left( i\hbar\frac{d\psi(\boldsymbol{x},t)}{dt} \right) e^{-i\boldsymbol{K}}(t)\cdot\boldsymbol{x} + \psi(\boldsymbol{x},t) e^{-i\boldsymbol{K}}(t)\cdot\boldsymbol{x} \left( \hbar\frac{d\boldsymbol{K}}{dt} \cdot \boldsymbol{x} \right)
\end{align*}

```

typst(542)

```

$
i planck dv(tilde(psi))(bold(x), t), t
&= i planck dv(, t) (psi(bold(x), t) e^(-i bold(K)(t) dot bold(x))) \
&= i planck [(dv(psi(bold(x), t), t) e^(-i bold(K)(t) dot bold(x))
+ psi(bold(x), t) dv(, t) (e^(-i bold(K)(t) dot t)))] \

```

```
&= (i planck dv(psi(bold(x),t),t)e^(-i bold(K)(t)dot bold(x)))
+ psi(bold(x),t)(i planck e^(-i bold(K)(t)dot bold(x))(-i dv(bold(K)(t),t)dot
bold(x)))\
&= (i planck dv(psi(bold(x),t),t))e^(-i bold(K)(t) dot bold(x))
+ psi(bold(x),t)e^(-i bold(K)(t)dot bold(x))(planck dv(bold(K)(t),t)dot bold(x))
$
```

300 文字以上セーブできました¹。

ちなみに tex がない typst の強みとして計算が可能な点が挙げられます。

```
#let char_tex = 875
#let char_typ = 542
#let reduce_rate = calc.round( (char_tex - char_typ)/char_tex * 100 , digits: 2)
#text(red)[texに比べてtypstでは#reduce_rate;%文字数が減りました。]
```

tex に比べて typst では 38.06% 文字数が減りました。

これがまた素晴らしいもので、ある程度書いたのちにどっか計算ミスが発覚！とかなった場合に関連する場所全て計算し直すとかしなくても大元の数字(今の例で言う char_tex とか)を変更してしまえば全ての箇所に変更が行き渡るのでとても助かる。calc.~と書くことで計算できる。どんな計算ができるかについては[こちら](#)を参照。

2.4 リンク

`#link("https://hoge hoge.jp", [リンク先])`で[リンク先](#)こうなる。リンクが下線+青字となっているのは再び main-template 内で

```
#show link: it => {
  underline[#text(blue, it)]
}
```

としてるから。ちなみに link と ref は別物なので、カスタムする際は一応注意。

2.5 reference

figure や数式に対する ref の方法は[節 2.2](#) , [節 2.3](#) にて既に触れた。そして今やったように、heading に対しても同様に ref できる。

```
== heading <tag_heading>
@tag_heading
```

英語にあわせてか、日本語であっても「章 1」、「節 1.1」のように出力される(章については show ルールを適用してあるから「第 1 章」となってるが)。気になるようならこれも main-template 内で supplement をいじろう。筆者は気にならなかったのもそのまま使っている。

参考文献についても適切に引用する必要がある。基本は同じで@に続けてタグを示してやればよい。今、references.bib に

```
@mastersthesis{bibref-tag,
  author = {東大太郎},
  school = {東京大学},
  title = {ぼくのかんがえたさいきょうのしゅうろんぶん},
  year = {2020},
  type={修士論文}
}
```

と書いてあるなら、文書内で@bibref-tag と書けば[\[1\]](#) といい感じに引用番号を振ってくれる。まあ、\ref{}だったのが@~になるだけなのでこの辺は分かりやすいだろう。

¹planck とか dv とかは tex の physics パッケージのように外部パッケージが必要。こちらでは physica というやつになる。詳しくは[節 3.2](#) へ。

2.6 par

段落に関する設定は `par` から。このテンプレートでは以下の設定を使っている。

```
#set par(  
  spacing: 0.65em,  
  first-line-indent: (amount: 1em, all: true),  
  justify: true,  
)
```

`first-line-indent` というのが段落始めにインデントを入れる設定。デフォルトでは英語文書に倣ってかセクション最初のパラグラフに関してはインデントしないようになっているが、`all: true`を入れることでこれを回避している。英語で書く場合にはこれを外しておこう。

2.7 page

ページ設定は `page` を使う。今の設定は以下

```
#set page(  
  paper: "a4",  
  margin: (top: 25mm, bottom: 10mm, inside: 25mm, outside: 15mm),  
  columns: 1,  
  numbering: "1",  
  number-align: top + right,  
  header: context {  
    if header_components.is_even_page() {  
      if header_components.is_chapter_page() {  
        header_components.header_even_h1()  
      } else {  
        header_components.header_even_noh1()  
      }  
    } else {  
      if header_components.is_chapter_page() {  
        header_components.header_odd_h1()  
      } else {  
        header_components.header_odd_noh1()  
      }  
    }  
  }  
  counter(footnote).update(0)  
,  
)
```

最終的に製本することを考えて余白を `inside/outside` で指定しており、対応して `header` もいろいろ変な設定をしている。不要と思うのであれば、余白は `left/right` で指定すればいいし `header` の設定もデフォルトに戻せばよい。

あと最後のは `footnote` をページごとに数字リセットするようにする設定。

第 3 章

tips

3.1 有用な関数

筆者が修論にて使用したカスタム関数のうち、ほかの人にも有用そうな奴らを紹介する。全部 `template.typ` の下の方にあるぞ。

3.1.1 mymemo, TBC

ただのブロック。mymemo は水色、TBC は橙で fill し、前者はただのメモとして使用し、後者は確認事項(要出典箇所とか、修正コメントもらった箇所とか)を視覚的にわかりやすくするのに使用。

```
#mymemo[mymemo]
#TBC[要確認事項]
```

mymemo

要確認事項

3.1.2 todo, show_todo

todo で文書内の各位置にタスクを記載すると、<todo>というタグのついたテキストとして出力する。そいでもって文書内の適当な場所にて `#show_todo` することにより、<todo>タグをクエリして一覧表示してくれる。あとついでに文書内の各 todo の箇所にリンクもつけてくれるよ。ラボで回覧してたくさんコメントをもらった時に対応し忘れることがなくなるのでとても便利。大抵概要を最後に書くと思うので、概要のページにて show するのがいいと思われる。

```
#todo()[やることその1]\
#todo()[やることその2]
#show: show_todo
```

やることその 1

やることその 2

To Do List:

[p.19: やることその 1](#)

[p.19: やることその 2](#)

筆者の普段の研究ノートではその発展系として status 引数を受け取るようにして、ステータスに応じて未着手/対応中/実行済の 3 段階に分けて表示するようにしている。

3.1.3 pbox

`#place` 関数というのがあり、こいつは文書内の好きな場所にコンテンツを配置できる。position 引数として top/horizon/bottom, left/center/right などが指定できるのだが、どうも内部的にはアラインメントも連動してしまうようで、例えば

```
#place(right)[
  #block(stroke: red, width: 50%)[
    Hello, World!
  ]
]
```

と書くと→このように表示されてしまう。

Hello, World!

`place(right)`を使いたい時ってのは単に右側に置きたいだけであって文字は普通に左詰の方がいい(たまに center とか right とかにしたいときもあるけどそれは別途指定するようにしたい)ので、こういうのを作った。といっても単に `place` の中で改めて `set align` しているだけである。適宜オプ

ションをつけてやることで block のカスタムが可能になる¹。

これを使えば、以下のように右配置でも左詰でテキストを書き込める。alignment: right と書けば右詰になるぞ。

```
#pbox(right, width: 90%, stroke: red)
[Hello, World!]
```

Hello, World!

3.1.4 smaller_list

デフォルトでは list は深さによらず全て同じ文字サイズで出力される。

- タイトル
- 本文
 - ▶ イントロ
 - 背景
 - 動機
 - ▶ 内容
 - hogehoge

人によっては深さによって文字サイズを小さくしたいと考えるだろう、筆者もスライド作る時にはよくそう思う。そういう時に使うもの。

```
#show list: smaller_list()
- タイトル
- 本文
  - イントロ
    - 背景
    - 動機
```

- タイトル
- 本文
 - ▶ イントロ
 - 背景
 - 動機

このように引数に何もつけなければデフォルトで 0.9em, 0.85em にする。level4 以降については無視。これを変更したければ普通に template の中でデフォルト値を書き換えるか set すればいいだけだが、スペースの問題などで一時的にもう少し小さく、あるいは大きくしたいときもあるだろう、ということで、引数として dd, ddd を受け取るようにしている。それぞれ depth2,3 の文字サイズを与える。

```
#show list: smaller_list(dd: 1.2em, ddd: 0.3em)
- タイトル
- 本文
  - イントロ
    - 背景
    - 動機
```

- タイトル
- 本文
 - ▶ イントロ
 - 背景
 - 動機

ちなみにリストには3つほどバリエーションがあつて、上に見せた番号なしリストの他番号つきリスト、説明付きリストがある。それぞれ対応する関数として #list, #enum, #terms というのがある

¹この挙動は、pbox の定義に引数として ..arg を受け取れるようにしてそれをそのまま block のオプションに投げられるようにすることで実現している。

```
#let pbox(pos, width: auto, dx: 0pt, dy: 0pt, alignment: left, ..arg, body) = {
place(pos, dx: dx, dy: dy)[
  #set align(alignment)
  #block(
    width: width,
    ..arg,
  )[#body]
]
```

ドット二つはその中身を展開することを表している。例えば pbox(right, stroke: red, fill: blue) と書くと arg としては “stroke: red, fill: blue” というそのままでは意味をなさない値が渡されるが、これをカンマ区切りで展開して “stroke: red”, “fill: blue” という別々の引数として block のオプションに流してくれる、ということ。

るが、最後のやつ以外は別にマークダウン形式の方で良いのではと思っている。せいぜいテンプレート作成時に set するくらい？

番号つきリスト

1. -を+に変えるだけ

2. 勝手にインクリメントしてくれる

1. インデントも同様

説明付きリスト

使い方 -を/に変えるだけ

使い所 いつ？筆者は使っていないしなんならこの説明書くためにドキュメント見直して知った

3.1.5 math-ul

ポスターの制作時に使った。下線を引くには `#underline()[hoge hoge]` でいけるが、こいつ数式モードに対しては働いてくれない。

`#underline()` [好きな公式は $E = mc^2$ です。] 好きな公式は $E = mc^2$ です。

(そもそも下線の位置が微妙でちょっと下にはみ出る文字(アルファベット使う時が顕著)につくときは線が途切れてしまいかなり不恰好。まあこれ自体は引数に `offset` というがあるので適切に set してやればよいのだが。)

ということで、数式モードに対しても同じように下線を引くのがこいつ。

`#math-ul()` [好きな公式は $E = mc^2$ です。] 好きな公式は $E = mc^2$ です。

まあ、数式に対して機能しない `underline` に変わって `box` を配置し、底辺にのみ `stroke` をつけただけです。卒論・修論においては使うことないと思うが、スライドやポスター制作においては少なくとも筆者的には重宝した。

3.2 有用なパッケージ

`main.typ` の冒頭にあるように、`#import` は自前の `typ` ファイルのみならず他パッケージも導入することができる。ここでは自分が使用した有用パッケージを紹介する。

ちなみにパッケージの一覧が [typst universe](https://typst.universe) にある。面白いものが多数あるので暇な時に眺めるといいだろう。ついでにいうと左のメニューから `templates` を選べば文書テーマの一覧が出てくる。今更ながら最初から律儀にカスタムしなくとも公式にあるテンプレート一覧から適当に好みを見つけて `import` して利用、慣れてきたらそれをカスタムしながら文法を覚えるという流れの方が多分よい。

3.2.1 unify, physica, fletcher

`unify` は単位、`physica` は物理関係のなんやら、`fletcher` はダイアグラム作成のためのパッケージ。https://qiita.com/key_271/items/8629980c4c1ff0e55f41 にいい感じにまとまった記事があるのでそちらを参照。

3.2.2 zero

<https://typst.app/universe/package/zero/>

数字のフォーマットをしてくれるパッケージ。 `num` と `format-table` をよく使った。

`num` は(`unify` にもあったが)誤差とか指数表記とかを書くための関数。`unify` のより若干やこっちの方が使いやすい印象がある。さすがに個人の好みレベルだが

Code	Output	Code	Output
<code>num("1.2e4")</code>	1.2×10^4	<code>num[1.2e4]</code>	1.2×10^4
<code>num("-5e-4")</code>	-5×10^{-4}	<code>num(fixed: -2)[0.02]</code>	0.02
<code>num("9.81+- .01")</code>	9.81 ± 0.01	<code>num("9.81+0.02- .01")</code>	$9.81^{+0.02}_{-0.01}$
<code>num("9.81+- .01e2")</code>	$(9.81 \pm 0.01) \times 10^2$	<code>num(base: 2)[3e4]</code>	3×2^4

てかオプションめっちゃあるんやね、知らなかった。num 自体の設定は set-num を使うらしい。

```
#set-num(product: math.dot, tight: true)
```

format-table は表内の数字位置を揃える。以下の例だと 1 列目は特に指定なく、2 列目に関しては小数点位置で揃えるように、ということ。

```
#figure({
  show: format-table(none, auto)
  table(
    columns: 2,
    [1], [1.2],
    [2], [2],
    [3], [300]
  )
})
```

1	1.2
2	2
3	300

3.2.3 cheq

<https://typst.app/universe/package/cheq/>

チェックリストを作れる。別になくても困らないものではあるが、たまに使ったので一応紹介。

```
#import "@preview/cheq:0.3.0": checklist
#show: checklist.with(extras: true)
- [x] タスク1
- [ ] タスク2
- [!] タスク3
- [u] タスク4
- [I] タスク5
- [>] タスク6
```

- ☑ タスク 1
- ☐ タスク 2
- ! タスク 3
- 🔼 タスク 4
- 💡 タスク 5
- ➡ タスク 6

Appendix

A

応用

本章では特に必須ではないが筆者が修論執筆において使用した、あるいはテンプレ整備時に見つけたテクニックを紹介する。

A.1 目次いじり

筆者は修論において目次を結構いじっていた。本テンプレではとりあえず色を出さずデフォルト設定を採用しているが、筆者と似たような拘りを持つ人がいた場合に参考になればという思いで、それらをまとめておく。

要件は以下の3点である:

- チャプターごとにスペースを入れる
- 目次において「第1章 タイトル 1」の点線は level 1 に対しては不要
 - ▶ こいつらはまあ、見やすさの問題
- 図目次、表目次において「図 1.1」とかじゃなく「1.1」と表示する
 - ▶ 図目次なんだから図なのは明確じゃん。

A.1.1 チャプターごとにスペース+点線一部削除

これは容易。outline へのエントリのうち level 1 であるものに対して次のように設定すれば良い。

```
#show outline.entry.where(level: 1): set block(above: 1.5em)
#show outline.entry.where(level: 1): set outline.entry(fill: none)
```

第1章 はじめに	1
1.1 how to use	1
1.2 ファイル構造	2
第2章 基本の記法	3
2.1 ==だと level2	3
2.2 figure	3
2.3 数式	8
2.4 リンク	10
2.5 reference	10
2.6 par	11
2.7 page	11
第3章 tips	13
3.1 有用な関数	13
3.2 有用なパッケージ	15
Appendix	18
A 応用	19
A.1 目次いじり	19
A.2 footnote いじり	21

B	dictionary	23
	謝辞	26
	参考文献	27

どう？結構スッキリしててよくない？筆者はこれが好み。

ちなみに今 level3 の heading が消えて表示されているのは、depth オプションを指定したから。

```
#outline(title: none, depth: 2)
```

A.1.2 図 1.1 → 1.1

これが結構大変だった。さっきの設定のまま同様に図目次を見せると状況がわかる。

図 1.1	preview のボタン位置	2
図 2.1	カービイじゃねえカービィだにどとまちがえるなくそが	4
頭 2.2	カービイじゃねえカービィだにどとまちがえるなくそが	6
図 2.3	本文用長いキャプション	6
図 2.4	placement: bottom	8
図 2.5	place+figure	8
図 A.1	図目次テスト用の図	22

うん、全部 level 1 だね。当然こんなスカスカの目次を作りたいわけないので、先ほどのようにチャプターごとに空白ってのをやりたければ自力でチャプターの切れ目を探してくる必要がある。いうてもチャプターが変わるごとにカウンタをリセットするようにしてあるので、下の番号が 1 かどうかを見ればよいのでこれは容易。

もう一点、「図 1.1」の「図」だけ外すというのが意外とめんどくさい。単に outline を表示する際に `set figure(supplement: none)` とすればいいように思うかもしれないが、outline は文書内にある figure をクエリして拾ってくるものなので今更 set したところで表示されるものは同じである。そのため entry に対して自ら show ルールを適用する他ない(と筆者は思っている)。そしてその際、チャプター番号は本文では数字、付録ではアルファベットなので、そこが本文か付録かの判定を行う必要がある。この判定は各 figure の位置するチャプターを拾ってきてそのナンバリングのタイプが function か str かを見ることによって可能¹。

実装は以下²。使いたければ `state.typ` で outline を実行する前にこの show ルールを適用すれば良い。

¹余談。当初は本文か付録かの判定に `is-app` という state 関数を使用していた。文書内の figure をクエリして各エントリとその直前の figure についてチャプター番号を取得して比較、減少した時に false から true に変更するという具合であった。実は筆者の修論ではこの手法が採用されていたのだが、本テンプレを作成するにあたり、

- 本文(の最後に figure が登場する章)が N 章
- 付録で figure が初めて登場するのが N 番目

である状態において先のチャプター番号比較を通過してしまい、例えば図 2.1 の次が図 B.1 であるときに B.1 でなく再び 2.1 になってしまうというバグがあることに気付いた。そんな謎構成の修論を書く人はきつといないだろうが、一応修正しておいた。

²余談その 2。outline の見せ方において link を使うようにしたが、link は main-template の中において青文字+下線で表示するようにしたのでここでは改めて `#show link: it => it.body` することで show をリセットした。実際の運用にあたっては、outline は main-template の前にのみ登場するのでここは不要である。

```
#show outline.entry: it => context {
  let loc = it.element.location()
  let current_chapter = query(heading.where(level: 1).before(loc)).last()
  let chapter_num = if type(current_chapter.numbering) == function {
    counter(heading).at(it.element.location()).at(0)
  } else if type(current_chapter.numbering) == str {
    let app_num = counter(heading).at(it.element.location()).at(0)
    numbering("A", app_num)
  } else {"panic!!"}
  let fignum = numbering("1.1", ..it.element.counter.at(loc))

  if fignum == "1" {v(1em)}
  link(it.element.location(), it.indented(
    [#chapter_num] + "." + [#fignum],
    it.inner()
  ))
}
#block(inset: 15pt)[
  #show link: it => it.body
  #outline(title: none, target: figure.where(kind: image))
]
```

1.1 preview のボタン位置	2
2.1 カービイじゃねえカービイだにどとまちがえるなくそが	4
2.2 カービイじゃねえカービイだにどとまちがえるなくそが	6
2.3 本文用長いキャプション	6
2.4 placement: bottom	8
2.5 place+figure	8
A.1 図目次テスト用の図	22

A.2 footnote いじり

どうでもいいが自分は footnote で結構遊んでいた。デフォルトだと¹こうなるが、ここは標準でいろいろ遊べる。例えば #set footnote(numbering: "㊦")とすれば²になるし、他にも #set footnote(numbering: "9")とすれば³になる。

自分の修論では、

```
#set footnote(numbering: (..arg) => {
  let daiji = ([壹], [弍], [参], [肆], [伍], [陸], [漆], [捌], [玖], [拾])
  let idx = arg.pos().at(0) - 1
  if idx > 9 { [拾] + daiji.at(idx - 10) } else { daiji.at(idx) }
})
```

として大字を用いていた^肆。

なんとこれを応用して、例えばインデックスにテキストでなく image を用いることだって可能。

¹数字

²丸数字

³何数字だよこれ

^肆ぶっちゃけ 1 ページに 10 個も脚注しねえし idx>9 の設定は不要ではある。

```
#set footnote(numbering: (..arg) => {  
  let image = (image("../fig/test.jpg", width: 20pt), [弐], [参], [肆], [伍], [陸], [漆],  
[捌], [玖], [拾])  
  let idx = arg.pos().at(0) - 1  
  image.at(idx)  
})
```

結果



キッショ。流石にこれは使うことないな。



図 A.1: 図目次テスト用の図

表 A.1: 表目次テスト用の表



あああ

B

dictionary

これが生きるタイプの修論だったというのもあるだろうが、筆者はこの dictionary の存在がとっても役に立ったので、紹介する。

以下のような形式で、key-value のマップを定義することが可能。以下の例を見れば大体使い方がわかるだろう。

```
#let dict = (
  name: "Typst",
  born: 2019,
)
#dict.name \
#(dict.launch = 20)
#dict.len() \
#dict.keys() \
#dict.values() \
#dict.at("born") \
#dict.insert("city", "Berlin")
#("name" in dict)
```

Typst
3
("name", "born", "launch")
("Typst", 2019, 20)
2019
true

さて、コレがどう役に立つのか。筆者の修論では、table に結果をまとめる際に大変役に立ちました。

chapter/data.typ に以下のように array, dictionary を定義する。実験・解析において考えられるカテゴリーやその結果の数字などをまとめたものである¹。

```
#let classes = (
  "Apple",
  "Banana",
  "Cherry",
  "Grape",
  "Orange",
)

#let model_A = (
  scores: (0.85, 0.92, 0.78, 0.88, 0.90),
  average: 0.866,
  total_accuracy: "87.1%"
)

#let model_B = (
  scores: (0.88, 0.90, 0.82, 0.85, 0.93),
  average: 0.876,
  total_accuracy: "88.5%"
)
```

そしてこれを用いて、次のように table を書くことができる。

```
#import "data.typ": *
#figure(
  table(
    columns: 3,
    stroke: none,
    ..(
```

¹gemini にいい感じの例となる適当なデータ群作ってって指示してできたデータです

```

    table.hline(y: 0),
    table.hline(y: 2, stroke: .5pt),
    table.hline(y: 8, stroke: (dash: "dashed")),
    table.hline(y: 9),
  ),
  align: (x, y) => if x == 0 { left } else { center },
  fill: (x, y) => {
    if y == 0 or y == 1 { gray.lighten(60%) } else if x == 0 { gray.lighten(80%) }
  },

  table.cell(rowspan: 2, align: horizon)[*Target Class*],
  table.cell(colspan: 2)[*Model Comparison (Precision)*],
  [*Model A (SVM)*], [*Model B (CNN)*],

  ..for (i, name) in classes.enumerate() {
    let row = (
      name,
      [#model_A.scores.at(i)],
      [#model_B.scores.at(i)],
    )

    if i == 2 {
      let subtotal = (
        [*Top-3 Average*],
        [#model_A.average*],
        [#model_B.average*],
      )
      row + subtotal
    } else {
      row
    }
  },

  [*Overall Accuracy*], model_A.total_accuracy, model_B.total_accuracy,
),
)

```

Target Class	Model Comparison (Precision)	
	Model A (SVM)	Model B (CNN)
Apple	0.85	0.88
Banana	0.92	0.9
Cherry	0.78	0.82
Top-3 Average	0.866	0.876
Grape	0.88	0.85
Orange	0.9	0.93
Overall Accuracy	87.1%	88.5%

ヘッダ書いてるところまでは普通の table である。本題はここから。

```

..for (i, name) in classes.enumerate() {
  let row = (
    name,

```

```

    [#model_A.scores.at(i)],
    [#model_B.scores.at(i)],
  )

```

classes という配列を順番に見ていき、i 番目を name に入れて{}内の処理をループする。そのループ内では一つの行に表示するものを定義している。1 列目に name そのもの、2,3 列目には dictionary model_A, model_B の scores の対応する場所を拾ってくるように、ということ。

```

if i == 2 {
  let subtotal = (
    [*Top-3 Average*],
    [#model_A.average*],
    [#model_B.average*],
  )
  row + subtotal
} else {
  row
}

```

そしてここで表示する際に特殊なルールを設けている。i==2 のとき、つまりデータ部分 3 行目を出力する際に追加で subtotal という別な配列を作って表示するということをしている。筆者は背景事象数の合計値の出力とかに使っていたな。

比較のため、ベタ書きした時の raw コードも見ておこう。

```

#table(
  columns: 3,
  stroke: none,
  .. (
    table.hline(y: 0),
    table.hline(y: 2, stroke: .5pt),
    table.hline(y: 8, stroke: dash: "dashed"),
    table.hline(y: 9),
  ),
  align: (x, y) => if x == 0 { left } else { center },
  fill: (x, y) => {
    if y == 0 or y == 1 { gray.lighten(60%) }
    else if x == 0 { gray.lighten(80%) }
  },

  table.cell(rowspan: 2, align: horizon)[*Target Class*],
  table.cell(colspan: 2)[*Model Comparison (Precision)*],
  [*Model A (SVM)*], [*Model B (CNN)*],

  [Apple], [0.85], [0.88],
  [Banana], [0.92], [0.90],
  [Cherry], [0.78], [0.82],

  [*Top-3 Average*], [*0.850*], [*0.866*],

  [Grape], [0.88], [0.85],
  [Orange], [0.90], [0.93],

  [*Overall Accuracy*], [87.1%], [88.5%]
)

```

まあそもそも typst だと table が書きやすいのでこれを修正する必要が出てきた、となったとていうほど面倒ではないが、これがもっと行・列が増える場合や、似たような表をたくさん書く必要がある場合にはその限りではないと思う。

謝辞

ここは謝辞。

なお、謝辞・参考文献については特に `#pagebreak(to: "odd")` はしてない。気になるようなら `state.typ` に書いておこう。

このテンプレート作成においては筆者の同期 O 氏に多大なる助力をいただきました、ここに感謝申し上げます。

(説明用。使用時は state.typ の mymemo ブロックを消してください)

参考文献は普通に.bib とか.yaml とかにまとめてから #bibliography で呼び出せばよい。style についてはいろいろあるだろうが、自分は“american-physics-society”でやった。どんなのが選べるかはドキュメント参照のこと。

あとここだけは英語設定にしないとうまいこといかない。日本語設定のままにすると例えば“S. Abe and T. Donald”が“S. Abe と T. Donald”って感じに出力されてしまう。

参考文献

- [1] 東大太郎, ぼくのかんがえたさいきょうのしゅうしろんぶん, 修士論文, 2020.